

Failure (prove the suite catches a regression) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (a real, repeatable process that also tells you *which* task caught the problem). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A test suite you've never watched fail is a suite you're trusting blindly — you don't know it would catch a real problem, you're only assuming it would.

The way to check: break something on purpose. Take a working prompt, remove one key instruction from it, and run your suite again. If the suite is doing its job, the score should visibly drop.

If nothing changes — if your suite still says everything passed — that's not good news. It means your suite has a hole, and you need to find and fix it before you can trust it.

Intermediate Version

This exercise is a **test of the tests** — like pressing the test button on a smoke alarm, instead of just trusting it works.

The process: record your suite's current score as a baseline. Pick one instruction in the prompt that matters (here: "always mention the 30-day return policy"). Delete that one instruction. Re-run the exact same suite, unchanged. Compare the new score to the baseline.

The exact pieces:

- **Keep the prompt in a file** (`practice/support_prompt.txt`), and read it fresh on every model call — then you can edit the file while the script waits, and the next run uses the new version.
- **A small, fixed task list** — 4 customer questions about returns, refunds and exchanges, each with the phrase a correct reply must contain (`"30"` , for the 30-day policy).
- `run_eval_suite()` returns `{task_id: True/False}` — one pass/fail per task, so you can see *which* task changed, not just the total.
- `input("...press Enter")` — pauses the script while you edit and save the prompt file.
- **Change exactly one thing.** If you change several things and the score drops, you won't know which change the suite caught.
- **Newly failed tasks** — the tasks that passed before and fail after. A `for` loop over the baseline results finds them.

Before you run anything, write down your *prediction*: which tasks should fail, and why?

Difference between Basic and Intermediate: Basic names the idea — break the prompt on purpose and watch the score drop. Intermediate turns it into a repeatable process: a prompt file read

fresh each call, a fixed task list, a baseline, exactly one change, and a per-task comparison that points at the task that caught it — or shows which task *should* have.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
support_prompt.txt: the system prompt, including
    "Always mention our 30-day return policy ..."

TASKS: 4 return/refund questions, each must_include "30"

run_eval_suite():
    for each task: ask the model, check must_include is in the reply
    return {task_id: True/False}

baseline = run_eval_suite()
pause: delete the 30-day line from the prompt file, press Enter
after = run_eval_suite()
compare how many passed before and after
```

Here's the part that makes the prompt file editable mid-run:

```
# practice/regression_catch_practice.py
def load_prompt():
    # read fresh every time, so an edit shows up on the next call
    with open("support_prompt.txt") as f:
        return f.read()
```

Intermediate Version

```
same TASKS and run_eval_suite()

baseline = run_eval_suite()
print baseline passed / total
input("remove the 30-day line, save, press Enter")
after = run_eval_suite()

newly_failed = []
for each task_id in baseline:
    if it passed in baseline and failed after: add it to newly_failed
```

```
if newly_failed: the suite caught it – print which tasks
else: the suite has a hole – tighten the rule that should have caught it
```

Here's the comparison — fill in the loop yourself:

```
# practice/regression_catch_practice.py
def find_newly_failed(baseline, after):
    newly_failed = []
    # your turn: loop over baseline; append task_id when
    # baseline[task_id] is True and after[task_id] is False
    ...
    return newly_failed
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)